Java Backend Interview Series

Chapter 1 · Sub-Chapter 1.4 (Companion)

Map Implementations Deep Dive

HashMap Internals · equals/hashCode · IdentityHashMap · Immutable Maps · Pitfalls

Baeldung-style Tutorial · 25+ Q&A · Import Blocks · hashCode contracts · ConcurrentHashMap advanced

1 Map Interface — Implementations Comparison

1.1 Introduction — Choosing the Wrong Map Costs You

The Java Collections Framework provides seven Map implementations, each optimized for a different scenario. Using HashMap when TreeMap is needed costs you sorted iteration. Using HashMap in a multithreaded context without synchronization causes data corruption. Using Hashtable in modern code saddles you with unnecessary overhead. This chapter is a companion deep-dive focusing on internals, contracts, pitfalls, and lesser-known implementations.

1.2 All Map Implementations — Side-by-Side

Implementation	Order	Thread Safe	Null Key	Null Value	Complexity	Backing
HashMap	None (random)	No	1 allowed	Yes	O(1) avg	Node[] + RB tree
LinkedHashMap	Insertion/Access	No	1 allowed	Yes	O(1) avg	HashMap + DLL
TreeMap	Sorted by key	No	No (NPE)	Yes	O(log n)	Red-Black tree
ConcurrentHashMap	None	Yes (CAS)	No (NPE)	No (NPE)	O(1) avg	Node[] + CAS
EnumMap	Enum ordinal	No	No (NPE)	Yes	O(1)	Object[] array
WeakHashMap	None	No	1 (weak ref)	Yes	O(1) avg	WeakReference keys
IdentityHashMap	None	No	Yes (==)	Yes	O(1) avg	Linear-probe array
Hashtable (legacy)	None	Yes (sync)	No (NPE)	No (NPE)	O(1) avg	Array + sync
Map.of() / Map.copyOf()	None	Read-only	No (NPE)	No (NPE)	O(1) avg	Compact array

TIP: In 99% of single-threaded code: HashMap. For concurrency: ConcurrentHashMap. For sorted: TreeMap. For enum keys: EnumMap. These four cover virtually every real use case.

2 HashMap Internals — Node[], hash(), Index Formula

2.1 Introduction — The Library Analogy

Imagine a library with 16 numbered shelves (buckets 0-15). When storing a book, the librarian runs a quick calculation on the ISBN (hashCode), XORs upper and lower bits (hash()), then takes the result modulo 16 ((n-1) & hash) to find the shelf. To retrieve: same calculation, direct to the shelf. No linear scan. This is exactly how HashMap works.

2.2 The Node Structure

```
import java.util.HashMap;
import java.util.Map;
// Simplified HashMap.Node (actual JDK source, Java 21)
static class Node<K,V> implements Map.Entry<K,V> {
final int hash; // cached hash of key
final K key;
V value;
Node<K,V> next; // null if no collision

Node(int hash, K key, V value, Node<K,V> next) {
this.hash = hash;
this.key = key;
this.value = value;
this.next = next;
}
}

// TreeNode (Java 8+) — extends LinkedHashMap.Entry extends Node
static final class TreeNode<K,V> extends LinkedHashMap.Entry<K,V> {
TreeNode<K,V> parent, left, right, prev;
boolean red;
// Used when chain grows to TREEIFY_THRESHOLD (8)
}
```

HashMap.Node and TreeNode (Java 8+)

2.3 hash() Spreading and Index Formula

```
import java.util.HashMap;
// Step 1: Get key's hashCode
int h = "hello".hashCode(); // e.g., 99162322
```

```
// Step 2: Spread bits — XOR upper 16 bits into lower 16 bits

// Purpose: keys that differ only in high bits still land in different buckets

int spread = h ^ (h >>> 16);

// Binary: 00000101111010000011110110010010

// XOR 00000000000000000000010111101000

// = 00000101111010000011100001111010

// Step 3: Select bucket using power-of-2 trick

int n = 16; // table.length (always power-of-2)

int index = (n - 1) & spread; // equivalent to spread % n, but faster

// (15) & spread = last 4 bits = bucket 0-15

// Why power-of-2 capacity?

// (n-1) & hash == hash % n ONLY when n is a power of 2

// Bitwise AND is much faster than modulo for power-of-2 divisors

// Also ensures resize simply checks one bit: entry stays or moves to index+oldCap
```

HashMap hash spreading and bucket index calculation

2.4 Resize — Rehashing Without Full Recalculation

```
import java.util.HashMap;
// When size > capacity * 0.75 -> double the table

// Old capacity: 16 (0b10000), new capacity: 32 (0b100000)

// Key insight: entry index in new table is EITHER

// same as old index, OR old index + old capacity

// Determined by ONE new bit: (oldCap & hash) == 0 -> same, else +oldCap

// Example: entry with hash = 0b...10101

// Old (n=16): (16-1) & hash = 0b01111 & 0b10101 = 0b00101 = 5

// New (n=32): (32-1) & hash = 0b11111 & 0b10101 = 0b10101 = 21 = 5+16

// Bit 5 (oldCap bit) of hash is 1 -> moves to 5+16=21

// This means Java 8 resize splits each chain into TWO chains:

// "lo" chain (bit not set) -> same index in new table

// "hi" chain (bit set) -> index + oldCap in new table

// Far faster than recomputing hash for every entry
```

HashMap resize: O(1) rehashing via single bit check

> WARNING: Initial capacity that is not a power-of-2 is rounded UP to the next power-of-2 by HashMap. Passing 10 gives capacity 16, not 10. Use Integer.highestOneBit(n-1)<<1 to understand what capacity you will actually get.

2.5 Treeification Step-by-Step

```
import java.util.HashMap;
import java.util.Map;
// Scenario: poor hashCode() causes all keys to collide in bucket 0
class BadKey {
int id;
BadKey(int id) { this.id = id; }
@Override public int hashCode() { return 42; } // always same bucket!
@Override public boolean equals(Object o) {
return o instanceof BadKey bk && bk.id == this.id;
}
}

Map<BadKey, String> map = new HashMap<>();
for (int i = 0; i < 20; i++) map.put(new BadKey(i), "v" + i);

// With capacity 64+ and chain length 8 in bucket 42:
// HashMap converts linked list -> TreeNode (Red-Black tree)
// get(key) now: O(log 20) ~= 4 comparisons, not O(20)

// TreeNode.find() compares by hash first, then equals()
// If neither distinguishes, uses Comparable.compareTo() if available
// Finally falls back to tieBreakOrder() (identity hash comparison)
```

Treeification triggered by hash collision

3 equals() and hashCode() Contract

3.1 Introduction — The Broken Map Key

The most common Map bug for beginners: storing a custom object as a key and never being able to retrieve it. The reason is almost always a missing hashCode() override. Without it, two logically equal objects hash to different buckets, so get() never finds the entry even though equals() would return true.

3.2 The Contract — Four Rules

Rule	Description	Consequence if Broken
Reflexivity	a.equals(a) must be true	Object cannot be found in any Collection
Symmetry	a.equals(b) => b.equals(a)	TreeSet/HashSet may contain duplicates
Transitivity	a==b, b==c => a==c	Violation of mathematical equivalence
Consistency	repeated calls return same result	Map entries silently lost on mutation
hashCode contract 1	a.equals(b) => a.hashCode()==b.hashCode()	get() misses equal keys in different buckets
hashCode contract 2	!a.equals(b) => hashCodes MAY differ	Collision only (performance, not correctness)

3.3 Bad vs Good hashCode()

```
import java.util.HashMap;
import java.util.Map;
// PROBLEM: class without hashCode override
class ProductId {
private final String code;
ProductId(String code) { this.code = code; }
@Override public boolean equals(Object o) {
return o instanceof ProductId p && p.code.equals(this.code);
}
// Missing hashCode! Uses Object.hashCode() = identity hash
}

Map<ProductId, String> inventory = new HashMap<>();
inventory.put(new ProductId("SKU-001"), "Widget");

// BUG: new ProductId("SKU-001") has DIFFERENT identity hash!
System.out.println(inventory.get(new ProductId("SKU-001"))); // null!
// equals() would return true, but different bucket -> never found
```

BEFORE: Missing hashCode causes Map to lose entries

```
import java.util.Objects;
```

```
import java.util.HashMap;
import java.util.Map;
// SOLUTION 1: Objects.hash() — simple, readable
class ProductId {
private final String code;
private final String category;

ProductId(String code, String category) {
this.code = code;
this.category = category;
}

@Override public boolean equals(Object o) {
if (this == o) return true;
if (!(o instanceof ProductId p)) return false;
return Objects.equals(code, p.code)
&& Objects.equals(category, p.category);
}

@Override public int hashCode() {
return Objects.hash(code, category); // 31-based polynomial
}
}

// SOLUTION 2: record — equals+hashCode auto-generated
record ProductId(String code, String category) {}

// SOLUTION 3: Manual — best performance for hot paths
@Override public int hashCode() {
int result = 31 + (code == null ? 0 : code.hashCode());
result = 31 * result + (category == null ? 0 : category.hashCode());
return result;
}

// Now it works correctly:
Map<ProductId, String> inventory = new HashMap<>();
inventory.put(new ProductId("SKU-001","Electronics"), "Widget");
System.out.println(inventory.get(new ProductId("SKU-001","Electronics"))); // Widget
```

AFTER: Correct hashCode — Objects.hash(), record, or manual

3.4 hashCode Quality — Distribution Matters

```
import java.util.Objects;
// BAD: constant hashCode — all keys in one bucket -> O(n)
@Override public int hashCode() { return 42; }

// BAD: only uses one field when equality uses two
// violates: a.equals(b) => same hashCode
@Override public int hashCode() { return code.hashCode(); }
// If equals() also checks category, two objects with same code
// but different category may be unequal but have same hash -> OK
// But if category is in equals and hashCode omits it, WRONG!

// GOOD: 31-based polynomial hash (Joshua Bloch's Effective Java recipe)
// 31 is prime; (31 * x) can be optimized by JIT as (x << 5) - x
@Override public int hashCode() {
int h = 17; // non-zero prime seed
h = 31 * h + f1; // int field
h = 31 * h + (f2 != null ? f2.hashCode() : 0); // Object field
h = 31 * h + Boolean.hashCode(flag);
h = 31 * h + Double.hashCode(ratio);
return h;
}

// BEST for new code: use record
record Point(int x, int y) {} // equals and hashCode generated correctly
```

hashCode quality — bad, good, and best patterns

> WARNING: Never include mutable fields in hashCode() if the object will be used as a Map key. Mutating a key after insertion changes its bucket and the entry is permanently lost.

4 Bad vs Good hashCode — Objects.hash(), IDE-gen, Manual

4.1 The Three Tiers of hashCode Implementations

In practice, developers write hashCode in three ways: delegating to Objects.hash() (simplest, slight boxing overhead for primitives), using IDE-generated code (good, explicit), or hand-rolling (fastest, most error-prone). Records eliminate the choice for simple value types.

4.2 Comparison of hashCode Strategies

Strategy	Code	Pros	Cons
record	record Point(int x, int y){}	Auto-generated, correct, zero code	Only for simple value types
Objects.hash()	return Objects.hash(f1,f2)	One-liner, null-safe	Boxes primitives (autoboxing)
IDE-generated	if/else chain of 31*h	Explicit, no boxing	Verbose, breaks on refactor
@EqualsAndHashCode (Lombok)	Annotation	No boilerplate, regenerates	Lombok dependency
Manual 31-prime	31*h+field pattern	Fastest, no allocation	Error-prone, verbose

4.3 Practical Examples

```
import java.util.Objects;
// Tier 1: record — zero effort, correct
record OrderKey(String orderId, int revision) {}
// equals() compares orderId and revision; hashCode() uses both

// Tier 2: Objects.hash() — one liner
class Customer {
String email; int age;
@Override public int hashCode() {
return Objects.hash(email, age); // age boxed to Integer
}
@Override public boolean equals(Object o) {
if (!(o instanceof Customer c)) return false;
return age == c.age && Objects.equals(email, c.email);
}
}

// Tier 3: Manual — no boxing, maximum speed
class HotKey {
int x, y, z; // primitives, no boxing in manual approach
```

```
@Override public int hashCode() {
int h = 17;
h = 31 * h + x;
h = 31 * h + y;
h = 31 * h + z;
return h;
}
@Override public boolean equals(Object o) {
if (this == o) return true;
if (!(o instanceof HotKey k)) return false;
return x == k.x && y == k.y && z == k.z;
}
}

// Verify contract with a quick test
OrderKey a = new OrderKey("ORD-1", 1);
OrderKey b = new OrderKey("ORD-1", 1);
System.out.println(a.equals(b)); // true
System.out.println(a.hashCode()==b.hashCode()); // true (contract satisfied)
```

Three tiers of hashCode implementations

TIP: Starting in Java 16, prefer records for value objects. Records auto-generate correct equals() and hashCode() based on all components, and are immutable by default — making them ideal Map keys.

5 ConcurrentHashMap Advanced — atomicity, streams, size()

5.1 Introduction

ConcurrentHashMap in Java 8+ is not just a thread-safe HashMap. It provides atomic bulk operations (compute, merge, forEach with parallelism threshold), aggregation methods (reduce, search, mapTo*), and a carefully designed size() approximation using Striped64. Understanding these is essential for writing high-performance concurrent code.

5.2 computeIfAbsent Thread Safety — A Subtle Bug

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.ArrayList;
import java.util.List;
// PROBLEM: putIfAbsent can wastefully create objects
ConcurrentHashMap<String, List<String>> map = new ConcurrentHashMap<>();

// Thread 1 and Thread 2 both execute simultaneously:
map.putIfAbsent("key", new ArrayList<>());
// Both threads create ArrayList BEFORE calling putIfAbsent
// One ArrayList is thrown away — wasted allocation
// For expensive objects (DB connections, etc.) this is a real cost

// SOLUTION: computeIfAbsent — lambda called AT MOST ONCE
map.computeIfAbsent("key", k -> new ArrayList<>());
// JDK guarantees: if two threads race, only one lambda executes
// The other thread gets the winner's result

// WARNING: do NOT call map methods inside computeIfAbsent lambda!
// This causes deadlock (the bucket lock is held during fn execution)
map.computeIfAbsent("outer", k -> {
map.put("inner", "value"); // DEADLOCK in Java 8 (fixed in Java 9+)
return "result";
});
```

computeIfAbsent thread safety and the nested-call deadlock

5.3 Bulk Operations with Parallelism Threshold

```
import java.util.concurrent.ConcurrentHashMap;
ConcurrentHashMap<String, Integer> scores = new ConcurrentHashMap<>();
scores.put("Alice",95); scores.put("Bob",82); scores.put("Carol",91);

// forEach with parallelismThreshold
```

```
// If map.size() < threshold: runs sequentially

// If map.size() >= threshold: uses ForkJoinPool.commonPool()

scores.forEach(2, (k, v) ->

System.out.println(Thread.currentThread().getName() + ": " + k));

// search — returns first non-null result (order undefined)

String topScorer = scores.search(1,

(k, v) -> v >= 90 ? k : null);

// "Alice" or "Carol" (whichever thread finds first)

// reduce — parallel aggregation

int totalScore = scores.reduceValues(1,

Integer::sum);

// reduceEntries — transform then reduce

int highScoreSum = scores.reduceEntries(1,

e -> e.getValue() >= 90 ? e.getValue() : 0,

Integer::sum);
```

ConcurrentHashMap bulk operations with parallelism threshold

5.4 size() Approximation — sumCount() and Striped64

ConcurrentHashMap.size() calls sumCount() which returns baseCount + sum(counterCells[i].value). The CounterCell array uses @Contended (CPU cache-line padding) to prevent false sharing. Under low contention, only baseCount is updated via CAS. Under high contention, CAS on baseCount fails and the thread updates a random CounterCell instead. This distributes the write load across the array, achieving near-linear scalability. The size is eventually consistent, not exact.

```
import java.util.concurrent.ConcurrentHashMap;

import java.util.concurrent.atomic.LongAdder;

ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

// size() is an approximation — not synchronized

// This is INTENTIONAL: a globally consistent size would require

// stopping all writers (like a stop-the-world event)

int approxSize = map.size(); // fast but approximate

long exactSize = map.mappingCount(); // long, also approximate but better API

// isEmpty() is also approximate (checks baseCount + first counterCell)

boolean empty = map.isEmpty();

// For counting semantics: use LongAdder (same Striped64 design)

import java.util.concurrent.atomic.LongAdder;

LongAdder counter = new LongAdder();
```

```
map.forEach(1, (k, v) -> counter.increment());

long count = counter.sum();
```

size() approximation and LongAdder pattern

WARNING: Never rely on ConcurrentHashMap.size() for critical decisions in concurrent code. The count may be stale by the time you use it. Use atomic guards (computeIfAbsent, putIfAbsent) instead of check-then-act patterns.

6 IdentityHashMap — Reference Equality for Object Graphs

6.1 Introduction — When equals() Is The Wrong Question

IdentityHashMap uses == (reference equality) and System.identityHashCode() instead of equals() and hashCode(). Two distinct objects that are logically equal (equals()=true) are treated as different keys. This sounds wrong for most use cases — but is exactly right for object graph traversal, proxy detection, and serialization where you care about object identity, not logical equality.

6.2 IdentityHashMap vs HashMap

```
import java.util.HashMap;
import java.util.IdentityHashMap;
import java.util.Map;
String a = new String("hello"); // force separate object
String b = new String("hello"); // same value, different reference

// HashMap: uses equals() — treats a and b as the SAME key
Map<String, Integer> hashMap = new HashMap<>();
hashMap.put(a, 1);
hashMap.put(b, 2); // overwrites! a.equals(b) is true
System.out.println(hashMap.size()); // 1

// IdentityHashMap: uses == — treats a and b as DIFFERENT keys
Map<String, Integer> identityMap = new IdentityHashMap<>();
identityMap.put(a, 1);
identityMap.put(b, 2); // different key (different reference)
System.out.println(identityMap.size()); // 2

// get uses == too
System.out.println(identityMap.get(a)); // 1
System.out.println(identityMap.get(new String("hello"))); // null!
```

IdentityHashMap vs HashMap: == vs equals()

6.3 Real Use Case — Object Graph Traversal (Cycle Detection)

```
import java.util.IdentityHashMap;
import java.util.Map;
// Detecting already-visited nodes in an object graph
// (e.g., serialization, deep copy, cycle detection)

public <T> T deepCopy(T root) {
// IdentityHashMap: track WHICH INSTANCES we have already copied
```

```
// Not which VALUES — two distinct equal objects need separate copies!
Map<Object, Object> visited = new IdentityHashMap<>();
return deepCopyHelper(root, visited);
}

@SuppressWarnings("unchecked")
private <T> T deepCopyHelper(T obj, Map<Object,Object> visited) {
if (obj == null) return null;
if (visited.containsKey(obj)) return (T) visited.get(obj); // cycle!

T copy = shallowCopy(obj); // creates new instance
visited.put(obj, copy); // register by IDENTITY before recursing

// ... recurse into fields ...
return copy;
}

// Other real use cases for IdentityHashMap:
// 1. Java serialization (ObjectOutputStream uses it internally)
// 2. Proxy/AOP frameworks (track proxied instances)
// 3. ClassLoader-aware caches (same class name, different loaders)
// 4. Permission/capability tracking by object identity
```

IdentityHashMap for object graph traversal and cycle detection

TIP: IdentityHashMap uses linear probing (open addressing) — no Node chains. It typically uses double the capacity of expected entries. This makes it memory-efficient for maps with few entries but a source of performance degradation if over-filled.

7 Immutable Maps — Map.of(), Map.copyOf(), unmodifiableMap()

7.1 Introduction — Defensive Copying and Truly Immutable Maps

Java provides three ways to produce a map that prevents external modification, but they have very different semantics. Map.of() and Map.of(k1,v1,...) (Java 9+) create a truly immutable compact map. Map.copyOf() (Java 10+) copies an existing map into an immutable one. Collections.unmodifiableMap() wraps an existing map but the backing map can still be mutated through the original reference.

7.2 Map.of() — Compact Immutable Map (Java 9)

```
import java.util.Map;
// Map.of() — up to 10 key-value pairs
Map<String, Integer> scores = Map.of(
"Alice", 95,
"Bob", 82,
"Carol", 91
);

// For more than 10 entries:
Map<String, Integer> large = Map.ofEntries(
Map.entry("Alice", 95),
Map.entry("Bob", 82),
Map.entry("Carol", 91),
Map.entry("Dave", 78)
// ... up to Integer.MAX_VALUE entries
);

// Characteristics:
// 1. UnsupportedOperationException on any mutation (put, remove, clear)
// 2. NullPointerException on null keys OR null values
// 3. IllegalArgumentException on duplicate keys
// 4. Order NOT guaranteed (may be randomized for security)
// 5. Compact internal array (not Node[] — saves memory)

scores.put("Dave", 70); // UnsupportedOperationException!
Map.of("a", null); // NullPointerException!
Map.of("a", 1, "a", 2); // IllegalArgumentException — duplicate key!
```

Map.of() — compact immutable map (Java 9)

7.3 Map.copyOf() vs Collections.unmodifiableMap()

```
import java.util.HashMap;
import java.util.Map;
import java.util.Collections;
Map<String, Integer> mutable = new HashMap<>();
mutable.put("A", 1);
mutable.put("B", 2);

// Collections.unmodifiableMap — WRAPPER, NOT a copy
Map<String, Integer> unmod = Collections.unmodifiableMap(mutable);
unmod.put("C", 3); // UnsupportedOperationException
mutable.put("C", 3); // This WORKS and changes are visible through unmod!
System.out.println(unmod.get("C")); // 3 (leaked mutation!)

// Map.copyOf() — DEEP defensive copy, truly immutable
Map<String, Integer> copy = Map.copyOf(mutable);
mutable.put("D", 4); // mutable changes
System.out.println(copy.get("D")); // null — copy is independent
copy.put("D", 4); // UnsupportedOperationException

// KEY DIFFERENCE:
// unmodifiableMap: O(1) wrap, mutation visible through original ref
// Map.copyOf(): O(n) copy, truly frozen, null-hostile
```

Map.copyOf() vs Collections.unmodifiableMap() — critical difference

Property	Map.of()	Map.copyOf()	unmodifiableMap()
Source	Inline literal	Copy of existing map	Wraps existing map
Backed by original?	No	No	YES (wraps it)
Original changes visible?	N/A	No	Yes
Null keys/values	NPE	NPE	Depends on backing
Duplicate keys	IAE	IAE	Last write wins
Memory	Compact array	Compact array	Adds one wrapper object
Java version	Java 9+	Java 10+	Java 2+

WARNING: Collections.unmodifiableMap() does NOT make the map immutable — it only prevents mutation through that particular reference. If anyone holds the original mutable map reference, they can still modify it, and those changes are visible through the "unmodifiable" view.

8 Common Map Pitfalls — Mutable Keys, Nulls, Iteration Order

8.1 Introduction

Maps are deceptively simple to use but have several subtle pitfalls that cause hard-to-debug production bugs. The most common: mutating a key after insertion, assuming iteration order, ignoring null handling differences between implementations, and race conditions from non-atomic compound operations.

8.2 Pitfall 1 — Mutable Keys

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;
import java.util.ArrayList;
// BUG: using a mutable List as a Map key
Map<List<Integer>, String> map = new HashMap<>();
List<Integer> key = new ArrayList<>(List.of(1, 2, 3));
map.put(key, "original"); // stored in bucket hash([1,2,3])

key.add(4); // MUTATION: key is now [1,2,3,4]
// hash([1,2,3,4]) != hash([1,2,3]) -> different bucket
System.out.println(map.get(key)); // null! Entry lost in old bucket
System.out.println(map.size()); // 1 (entry still exists but unreachable)

// FIX: use immutable key
Map<List<Integer>, String> safe = new HashMap<>();
safe.put(List.of(1, 2, 3), "safe"); // List.of() is immutable
// OR: copy to an immutable snapshot before using as key
List<Integer> snapshot = List.copyOf(key);
safe.put(snapshot, "snapshot");
```

Pitfall: mutable key causes Map entry to become unreachable

8.3 Pitfall 2 — Null Handling Differences

```
import java.util.HashMap;
import java.util.TreeMap;
import java.util.concurrent.ConcurrentHashMap;
import java.util.Map;
import java.util.Optional;
// HashMap: one null key allowed, null values allowed
HashMap<String, String> hashMap = new HashMap<>();
hashMap.put(null, "null key"); // OK
```

```
hashMap.put("key", null); // OK

// TreeMap: null key throws NullPointerException (Comparable.compareTo(null))
TreeMap<String, String> treeMap = new TreeMap<>();
treeMap.put(null, "value"); // NullPointerException!

// ConcurrentHashMap: null key AND null value throw NullPointerException
ConcurrentHashMap<String, String> chm = new ConcurrentHashMap<>();
chm.put(null, "value"); // NullPointerException!
chm.put("key", null); // NullPointerException!

// Safe pattern: use Optional<V> for nullable values
Map<String, Optional<String>> safeMap = new ConcurrentHashMap<>();
safeMap.put("key", Optional.of("value"));
safeMap.put("nullKey", Optional.empty()); // represents absent/null
```

Null handling varies across Map implementations

8.4 Pitfall 3 — Iteration Order Assumptions

```
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.TreeMap;
import java.util.Map;
// BUG: assuming HashMap has consistent iteration order
Map<String, Integer> map = new HashMap<>();
map.put("C", 3); map.put("A", 1); map.put("B", 2);
map.forEach((k,v) -> System.out.print(k)); // could be ANY order
// Output varies between JVM runs, Java versions, JVM implementations

// BUG: assuming Map.of() preserves insertion order
Map<String,Integer> m = Map.of("C",3,"A",1,"B",2);
m.forEach((k,v) -> System.out.print(k)); // order undefined and RANDOMIZED

// FIX: use LinkedHashMap for insertion-order guarantee
Map<String, Integer> ordered = new LinkedHashMap<>();
ordered.put("C", 3); ordered.put("A", 1); ordered.put("B", 2);
ordered.forEach((k,v) -> System.out.print(k)); // always C A B

// FIX: use TreeMap for sorted order
Map<String, Integer> sorted = new TreeMap<>(map);
sorted.forEach((k,v) -> System.out.print(k)); // always A B C
```

Pitfall: never assume HashMap or Map.of() iteration order

8.5 Pitfall 4 — Non-Atomic Compound Operations

```
import java.util.concurrent.ConcurrentHashMap;
// BUG: check-then-act is not atomic on HashMap or ConcurrentHashMap
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

// Thread 1 and Thread 2 both execute this:
if (!map.containsKey("counter")) { // Thread 1: false (not present)
map.put("counter", 1); // Thread 2: also false -> also puts 1
} // Result: race, value reset to 1

// FIX: use atomic compute
map.merge("counter", 1, Integer::sum); // always atomic

// Another common bug: increment is not atomic
map.put("counter", map.get("counter") + 1); // get + put not atomic!

// FIX:
map.compute("counter", (k, v) -> v == null ? 1 : v + 1); // atomic
```

Pitfall: non-atomic compound operations in concurrent Maps

> WARNING: On ConcurrentHashMap, each individual method is atomic, but COMBINATIONS of methods are not. containsKey() followed by put() is a classic TOCTOU race. Always use compute(), merge(), or putIfAbsent() for atomic compound operations.

9 Advanced Map Patterns — Multimap, BiMap & Computed Caches

9.1 Introduction

The JDK provides only single-valued maps. Real applications frequently need: Multimaps (one key -> multiple values), BiMaps (bidirectional key-value lookup), and computed caches (lazy populate on first access). This section shows how to implement these patterns with standard JDK classes before reaching for Guava.

9.2 Multimap Pattern — Key to Multiple Values

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;
import java.util.concurrent.ConcurrentHashMap;
// Pattern: Map<K, List<V>> as a Multimap
Map<String, List<String>> multimap = new HashMap<>();

// Adding values — use computeIfAbsent for clean code
multimap.computeIfAbsent("fruits", k -> new ArrayList<>()).add("apple");
multimap.computeIfAbsent("fruits", k -> new ArrayList<>()).add("banana");
multimap.computeIfAbsent("vegs", k -> new ArrayList<>()).add("carrot");

// Retrieving values — never returns null
List<String> fruits = multimap.getOrDefault("fruits", List.of());

// Removing one value from a key
multimap.computeIfPresent("fruits", (k, list) -> {
list.remove("apple");
return list.isEmpty() ? null : list; // null return removes the key
});

// Size of all values across all keys
long totalValues = multimap.values().stream()
.mapToLong(List::size).sum();

// Thread-safe multimap
Map<String, List<String>> safeMultimap = new ConcurrentHashMap<>();
safeMultimap.computeIfAbsent("key",
k -> Collections.synchronizedList(new ArrayList<>())).add("val");
```

Multimap pattern using Map> + computeIfAbsent

9.3 BiMap Pattern — Bidirectional Lookup

```
import java.util.HashMap;
import java.util.Map;
// BiMap: both key->value and value->key lookups must be O(1)
// JDK has no built-in BiMap; implement with two HashMaps
class BiMap<K, V> {
private final Map<K, V> forward = new HashMap<>();
private final Map<V, K> backward = new HashMap<>();

public void put(K key, V value) {
if (forward.containsKey(key)) {
backward.remove(forward.get(key)); // remove old reverse mapping
}
if (backward.containsKey(value)) {
forward.remove(backward.get(value)); // remove old forward mapping
}
forward.put(key, value);
backward.put(value, key);
}

public V get(K key) { return forward.get(key); }
public K getKey(V value) { return backward.get(value); }
public boolean containsKey(K key) { return forward.containsKey(key); }
public boolean containsValue(V v) { return backward.containsKey(v); }
}

// Usage
BiMap<String, Integer> codeToId = new BiMap<>();
codeToId.put("US", 1); codeToId.put("GB", 44); codeToId.put("DE", 49);
System.out.println(codeToId.get("US")); // 1
System.out.println(codeToId.getKey(44)); // GB
```

BiMap: bidirectional lookup with two HashMaps

9.4 Computed Cache — Memoization with computeIfAbsent

```
import java.util.HashMap;
import java.util.Map;
// Fibonacci memoization using computeIfAbsent
Map<Integer, Long> memo = new HashMap<>();
```

```
// Note: Cannot use computeIfAbsent recursively in Java 8
// (ConcurrentModificationException risk on HashMap)
// Safe approach: check-then-put pattern
public long fib(int n) {
if (n <= 1) return n;
Long cached = memo.get(n);
if (cached != null) return cached;
long result = fib(n-1) + fib(n-2);
memo.put(n, result);
return result;
}

// Non-recursive: safe with computeIfAbsent
// Pre-warm the cache with iteration, then computeIfAbsent for lookup
for (int i = 2; i <= 50; i++) {
final int n = i;
memo.put(n, memo.get(n-1) + memo.get(n-2));
}
// Now safe: memo.computeIfAbsent(n, k -> memo.get(k-1)+memo.get(k-2))
// won't recurse into computeIfAbsent again
```

Memoization cache — safe patterns with HashMap

TIP: For production memoization caches, use Caffeine's Cache<K,V> with CacheLoader. It handles concurrent access, max size, and expiry correctly without rolling your own solution.

11 Custom HashMap from Scratch — Interview Classic

11.1 Introduction — Why Implement One?

Interviewers at top tech companies (FAANG) often ask candidates to implement a simplified HashMap from scratch. This tests understanding of hashing, collision handling, load factor, and dynamic resizing. This section walks through a clean implementation with step-by-step explanation.

11.2 Minimal HashMap Implementation

```
import java.util.HashMap;
import java.util.Map;
@SuppressWarnings("unchecked")
public class MyHashMap<K, V> {
private static final int DEFAULT_CAPACITY = 16;
private static final float DEFAULT_LOAD_FACTOR = 0.75f;

private Object[][] table; // each slot: [key, value, next...]
// Simpler: use Node
static class Node<K,V> {
final int hash; K key; V value; Node<K,V> next;
Node(int hash,K k,V v,Node<K,V> n){hash=hash;key=k;value=v;next=n;}
}

private Node<K,V>[] buckets;
private int size;
private final float loadFactor;

public MyHashMap() { this(DEFAULT_CAPACITY, DEFAULT_LOAD_FACTOR); }

public MyHashMap(int cap, float lf) {
buckets = new Node[cap]; loadFactor = lf;
}

private int hash(K key) {
if (key == null) return 0;
int h = key.hashCode();
return (h ^ (h >>> 16)) & (buckets.length - 1);
}

public void put(K key, V value) {
int idx = hash(key);
```

```
Node<K,V> head = buckets[idx];
for (Node<K,V> n = head; n != null; n = n.next) {
if (n.hash==idx && (n.key==key||(key!=null&&key.equals(n.key)))){
n.value = value; return; // update
}
}
buckets[idx] = new Node<>(idx, key, value, head); // prepend
if (++size > buckets.length * loadFactor) resize();
}

public V get(K key) {
int idx = hash(key);
for (Node<K,V> n = buckets[idx]; n != null; n = n.next) {
if (n.key==key || (key!=null && key.equals(n.key))) return n.value;
}
return null;
}

private void resize() {
Node<K,V>[] old = buckets;
buckets = new Node[old.length * 2]; size = 0;
for (Node<K,V> head : old)
for (Node<K,V> n = head; n != null; n = n.next)
put(n.key, n.value);
}
public int size() { return size; }
}
```

Minimal HashMap implementation — interview-ready

> TIP: In real interviews, start with the data structure (Node array), then implement put() with collision handling, then get(), then resize(). Mention treeification (Java 8) as a known optimization you would add for production.

11.3 Testing the Custom HashMap

```
// No additional imports needed — uses MyHashMap above
MyHashMap<String, Integer> map = new MyHashMap<>();
map.put("alice", 95);
map.put("bob", 82);
map.put("alice", 98); // update existing
map.put(null, 0); // null key (bucket 0)
```

```
System.out.println(map.get("alice")); // 98

System.out.println(map.get("bob")); // 82

System.out.println(map.get("carol")); // null (not found)

System.out.println(map.get(null)); // 0

System.out.println(map.size()); // 3


// Test collision handling: force all keys to bucket 0

for (int i = 0; i < 20; i++) {

map.put("key" + i, i); // distributed across buckets

}

System.out.println(map.size()); // 23 (3 + 20)
```

Testing the custom HashMap implementation

12 Interview Patterns — Map in Algorithm Problems

12.1 Introduction

Maps appear in a huge fraction of algorithm interview problems. The most common patterns: frequency counting, grouping, two-sum/complement lookup, sliding window with character count, and graph adjacency representation. Mastering these patterns with the right Map implementation is essential.

12.2 Two-Sum Pattern — O(n) with HashMap

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;
import java.util.ArrayList;
import java.util.Set;
import java.util.HashSet;
// Classic two-sum: find indices of two numbers that sum to target
public static int[] twoSum(int[] nums, int target) {
Map<Integer, Integer> seen = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
int complement = target - nums[i];
if (seen.containsKey(complement)) {
return new int[]{seen.get(complement), i};
}
seen.put(nums[i], i);
}
return new int[]{};
}

// Variant: two-sum returns all pairs (not indices)
public static List<int[]> twoSumAllPairs(int[] nums, int target) {
Map<Integer, Integer> count = new HashMap<>();
List<int[]> result = new ArrayList<>();
for (int n : nums) count.merge(n, 1, Integer::sum);
Set<Integer> visited = new HashSet<>();
for (int n : nums) {
int c = target - n;
if (!visited.contains(n) && count.containsKey(c)) {
if (c != n || count.get(n) > 1) result.add(new int[]{n, c});
```

```
}
visited.add(n);
}
return result;
}
```

Two-sum pattern — O(n) with HashMap complement lookup

12.3 Sliding Window Character Count with HashMap

```
import java.util.HashMap;
import java.util.Map;
// Minimum window substring: find smallest s window containing all t chars
public static String minWindow(String s, String t) {
Map<Character, Integer> need = new HashMap<>();
Map<Character, Integer> window = new HashMap<>();

for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

int left = 0, right = 0, valid = 0;
int start = 0, minLen = Integer.MAX_VALUE;

while (right < s.length()) {
char c = s.charAt(right++);
if (need.containsKey(c)) {
window.merge(c, 1, Integer::sum);
if (window.get(c).equals(need.get(c))) valid++;
}
while (valid == need.size()) { // shrink window
if (right - left < minLen) {
start = left; minLen = right - left;
}
char d = s.charAt(left++);
if (need.containsKey(d)) {
if (window.get(d).equals(need.get(d))) valid--;
window.merge(d, -1, Integer::sum);
}
}
}
return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start+minLen);
}
```

```
// Test: minWindow("ADOBECODEBANC", "ABC") -> "BANC"
```

Minimum window substring — sliding window with two HashMaps

12.4 Graph Adjacency List with Map

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;
import java.util.ArrayList;
import java.util.Set;
import java.util.HashSet;
import java.util.Queue;
import java.util.ArrayDeque;
// Directed graph using Map<Node, List<Node>>
Map<String, List<String>> graph = new HashMap<>();

// Add edge: A -> B
graph.computeIfAbsent("A", k -> new ArrayList<>()).add("B");
graph.computeIfAbsent("A", k -> new ArrayList<>()).add("C");
graph.computeIfAbsent("B", k -> new ArrayList<>()).add("D");
graph.computeIfAbsent("C", k -> new ArrayList<>()).add("D");
graph.computeIfAbsent("D", k -> new ArrayList<>());

// BFS from a node
public static List<String> bfs(Map<String,List<String>> g, String start) {
List<String> order = new ArrayList<>();
Set<String> visited = new HashSet<>();
Queue<String> queue = new ArrayDeque<>();
queue.offer(start); visited.add(start);
while (!queue.isEmpty()) {
String node = queue.poll();
order.add(node);
for (String neighbor : g.getOrDefault(node, List.of())) {
if (visited.add(neighbor)) queue.offer(neighbor);
}
}
return order;
}
// bfs(graph, "A") -> [A, B, C, D]
```

Graph adjacency list using Map> with BFS

10 Coding Exercises

10.1 Exercise A — First Non-Repeating Character

Find the first character in a string that appears exactly once. Use LinkedHashMap to preserve order of first occurrence while tracking frequency.

```
import java.util.LinkedHashMap;
import java.util.Map;
// Solution using LinkedHashMap to preserve insertion order
public static char firstNonRepeating(String s) {
// LinkedHashMap: insertion order -> first occurrence order
Map<Character, Integer> count = new LinkedHashMap<>();
for (char c : s.toCharArray()) {
count.merge(c, 1, Integer::sum);
}
// Iterate in insertion order: first entry with count 1
for (Map.Entry<Character, Integer> e : count.entrySet()) {
if (e.getValue() == 1) return e.getKey();
}
return '\0'; // none found
}

// Test
System.out.println(firstNonRepeating("leetcode")); // l
System.out.println(firstNonRepeating("aabb")); // \0 (none)
System.out.println(firstNonRepeating("loveleet")); // v
```

Exercise A: first non-repeating char using LinkedHashMap

10.2 Exercise B — Verify hashCode/equals Contract

Write a utility method that verifies the hashCode/equals contract for a list of objects: every pair where equals() is true must have the same hashCode.

```
import java.util.List;
import java.util.ArrayList;
import java.util.Objects;
// Exercise B Solution
public static <T> List<String> findContractViolations(List<T> objects) {
List<String> violations = new ArrayList<>();
for (int i = 0; i < objects.size(); i++) {
for (int j = i+1; j < objects.size(); j++) {
```

```
T a = objects.get(i);
T b = objects.get(j);
boolean eq = Objects.equals(a, b);
boolean sameHash = a.hashCode() == b.hashCode();
if (eq && !sameHash) {
violations.add(String.format(
"equals=true but hashCodes differ: %s(%d) vs %s(%d)",
a, a.hashCode(), b, b.hashCode()));
}
}
}
return violations;
}

// Test with a broken class
class BrokenKey {
String name;
BrokenKey(String n) { this.name = n; }
@Override public boolean equals(Object o) {
return o instanceof BrokenKey b && b.name.equals(name);
}
// hashCode NOT overridden -> violation!
}
List<BrokenKey> keys = List.of(new BrokenKey("x"), new BrokenKey("x"));
System.out.println(findContractViolations(keys)); // [violation]
```

Exercise B: hashCode/equals contract verifier

10.3 Exercise C — Atomic Counter Map

Given N threads each incrementing 1000 keys by 1, implement a correct thread-safe frequency counter. Show the broken version and the correct version.

```
import java.util.Map;
import java.util.HashMap;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.LongAdder;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
// BROKEN: HashMap + simple increment (race condition)
Map<String, Integer> broken = new HashMap<>();
```

```
// Multiple threads doing: broken.put(k, broken.getOrDefault(k,0)+1)
// Result: lost updates! Final count < expected

// CORRECT Version 1: ConcurrentHashMap + merge
ConcurrentHashMap<String, Integer> correct1 = new ConcurrentHashMap<>();
Runnable task1 = () -> {
for (int i = 0; i < 1000; i++) {
correct1.merge("counter", 1, Integer::sum);
}
};

// CORRECT Version 2: ConcurrentHashMap + LongAdder (highest throughput)
ConcurrentHashMap<String, LongAdder> correct2 = new ConcurrentHashMap<>();
Runnable task2 = () -> {
for (int i = 0; i < 1000; i++) {
correct2.computeIfAbsent("counter", k -> new LongAdder()).increment();
}
};
// LongAdder: Striped64 design, extremely low CAS contention

// Run 10 threads for 3 seconds
ExecutorService ex = Executors.newFixedThreadPool(10);
for (int i = 0; i < 10; i++) ex.submit(task2);
ex.shutdown(); ex.awaitTermination(5, TimeUnit.SECONDS);
System.out.println(correct2.get("counter").sum()); // exactly 10000
```

Exercise C: correct thread-safe counter — merge() vs LongAdder

Cheat Sheet Quick Reference — Internals, Contracts & Pitfalls

CS.1 HashMap Internal Flow — put(k, v)

```
// put("hello", 42) step-by-step:

// 1. h = "hello".hashCode() = 99162322

// 2. spread = h ^ (h >>> 16) = 98847491 (XOR)

// 3. index = (table.length-1) & spread = bucket 3 (for cap=16)

// 4. If table[3] == null: insert new Node(hash,key,value,null)

// 5. If table[3] != null:

// a. Check head: hash matches AND (key==k OR key.equals(k))? update value

// b. If TreeNode: tree insert (O log n)

// c. Else: walk chain; if key found update; else append at tail

// d. If chain length >= TREEIFY_THRESHOLD (8) AND capacity>=64: treeify

// 6. ++modCount (fail-fast iterator invalidation)

// 7. If ++size > threshold (capacity*0.75): resize(2*capacity)
```

HashMap.put() internals — step by step

CS.2 equals/hashCode — 6 Rules Cheat Sheet

#	Rule	Code Implication
1	Reflexivity: a.equals(a)==true	Never return false for this==o
2	Symmetry: a.equals(b) <=> b.equals(a)	Tricky with inheritance; use instanceof
3	Transitivity: a==b,b==c => a==c	Avoid adding fields in subclass equals
4	Consistency: repeated calls same result	No random values in equals/hashCode
5	a.equals(b) => a.hashCode()==b.hashCode()	Include same fields in BOTH methods
6	Not required: !equals -> different hash	Collisions allowed; bad performance only

CS.3 Top 5 Map Pitfalls Summary

Pitfall	Symptom	Fix
Mutable key	get() returns null after mutation	Use immutable keys (String, record, final)
Missing hashCode	get() returns null always	Override both equals AND hashCode
Assuming HashMap order	Tests fail on JVM upgrade	Use LinkedHashMap or TreeMap for order
CHM null key/value	NullPointerException at runtime	Use Optional<V> or sentinel values

Pitfall	Symptom	Fix
Non-atomic CHM ops	Lost updates under concurrency	Use compute/merge/putIfAbsent

CS.4 Map.of() vs Map.copyOf() vs unmodifiableMap()

Feature	Map.of()	Map.copyOf()	unmodifiableMap()
Creates new map?	Yes	Yes	No (wrapper)
Source can still mutate?	N/A	No	YES — visible through wrapper
Null keys/values?	NPE	NPE	Depends on source
Java version	9+	10+	2+
Use when	Inline literals	Defensive copy	Read-only view (careful!)

Q&A Interview Questions — 25+ Standard & Advanced

Q1-Q12 Standard Questions

Q1. What are the seven Map implementations in the JDK?

HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap, EnumMap, WeakHashMap, IdentityHashMap. Additionally: Hashtable (legacy), and immutable maps from Map.of()/Map.copyOf().

Q2. What is the hashCode() contract with equals()?

If a.equals(b) is true, then a.hashCode() must equal b.hashCode(). The reverse is NOT required — unequal objects may share hashCodes (collision). Both must be consistent: repeated calls on unchanged objects return same result.

Q3. Why must you override both equals() and hashCode() together?

HashMap uses hashCode() to find the bucket and equals() to identify the exact entry. Without hashCode(), equal objects land in different buckets (never found). Without equals(), all objects in the same bucket are treated as distinct.

Q4. What is Objects.hash() and when should you use it?

Objects.hash(f1, f2, ...) calls Arrays.hashCode(new Object[]{f1,f2,...}) — a 31-based polynomial. Convenient for multi-field hashCode but boxes primitives. Use for non-performance-critical code; use manual 31*h+field for hot paths.

Q5. What is the difference between Map.of() and Map.copyOf()?

Map.of() creates a new immutable map from inline literals. Map.copyOf() copies an existing map into a new immutable map. Both forbid null keys/values. Map.copyOf() ensures independence from the original; Map.of() has no original.

Q6. What does Collections.unmodifiableMap() guarantee?

It prevents mutation through the returned reference only. The backing map can still be mutated through the original reference, and changes are visible through the unmodifiable view. It is NOT an immutable copy.

Q7. When should you use IdentityHashMap?

When you need == (reference equality) instead of equals() for keys: object graph traversal (deep copy, serialization), proxy detection, ClassLoader-keyed caches, or tracking specific object instances regardless of their logical equality.

Q8. What is the internal structure of IdentityHashMap?

IdentityHashMap uses a linear probing (open addressing) array, not chained nodes. It uses System.identityHashCode() for hashing. Typically allocated at 2x expected entries to reduce probe length. No TreeNode — just array slots.

Q9. What happens if you put a null key in TreeMap?

TreeMap throws NullPointerException when put(null, value) is called, because TreeMap needs to call compareTo() or the Comparator on the key, and null cannot be compared. This is true even with a custom Comparator unless it explicitly handles null.

Q10. What is the difference between Map.entry() and AbstractMap.SimpleEntry?

Map.entry(k,v) (Java 9+) creates an immutable Map.Entry (setValue throws UnsupportedOperationException). AbstractMap.SimpleEntry and SimpleImmutableEntry are older alternatives. Map.entry() is the modern idiom for Map.ofEntries().

Q11. How does ConcurrentHashMap.computeIfAbsent() guarantee the function runs at most once?

computeIfAbsent acquires a lock (synchronized on bucket head or uses CAS for empty bucket) before checking absence and calling the function. The lock is held for the entire check+compute sequence, ensuring only one thread's function executes.

Q12. What is false sharing and how does ConcurrentHashMap avoid it?

False sharing: two independent variables on the same CPU cache line cause cache ping-pong between cores. ConcurrentHashMap.CounterCell uses @Contended annotation (which adds 128-byte padding) so each CounterCell occupies a separate cache line.

Q13-Q25 Advanced Questions

Q13. What is the putIfAbsent race condition on ConcurrentHashMap?

putIfAbsent(k, new ExpensiveObject()) evaluates new ExpensiveObject() BEFORE calling the method. Two threads may both create the object; only one's result is stored, the other is discarded. Use computeIfAbsent(k, fn) to create the value lazily — fn runs at most once.

Q14. Explain how HashMap.resize() avoids full rehashing.

Because capacity is always a power-of-2, doubling it adds exactly one bit to the index mask. Each entry either stays at its old index (new bit is 0) or moves to old_index + old_capacity (new bit is 1). No hashCode() re-invocation is needed — just check the new bit of the cached hash.

Q15. Can two objects with different hashCodes be equal?

No. The contract states a.equals(b) => a.hashCode()==b.hashCode(). Therefore if hashCodes differ, objects CANNOT be equal. (The converse is not true: same hashCode does not imply equality.)

Q16. Why does HashMap allow one null key while ConcurrentHashMap does not?

HashMap handles null keys specially (hash=0, stored in bucket 0). ConcurrentHashMap cannot allow null because null return from get() would be ambiguous in concurrent code — it could mean absent or null-valued, requiring a separate containsKey() call that creates a TOCTOU race.

Q17. What is the difference between Map.of() order and TreeMap order?

Map.of() order is undefined and intentionally randomized between JVM runs (for hash-DoS protection). TreeMap order is the natural ordering of keys (Comparable.compareTo()) or the provided Comparator, consistent and deterministic.

Q18. How does WeakHashMap know when to clean up entries?

WeakHashMap keys are stored as WeakReference<K> objects registered with a ReferenceQueue. After GC, cleared references appear on the queue. WeakHashMap polls this queue in expungeStaleEntries() which is called from get(), put(), size(), and other public methods.

Q19. When is a HashMap entry treeified vs when does it resize?

If a bucket chain reaches TREEIFY_THRESHOLD (8) but table capacity < MIN_TREEIFY_CAPACITY (64): resize (double capacity) instead of treeify. If capacity >= 64 and chain >= 8: treeify. Rationale: small tables with long chains benefit more from wider distribution via resize than from tree overhead.

Q20. What is the time complexity of Map.copyOf()?

O(n) where n is the number of entries. Map.copyOf() iterates the source map and stores entries in a new compact internal array. The result has O(1) get/containsKey backed by a hash table implemented as a flat array.

Q21. How does IdentityHashMap handle hash collisions?

IdentityHashMap uses linear probing: if slot i is occupied, it tries i+2, i+4, ... (wrapping around). The step size of 2 keeps (key, value) pairs together (key at even index, value at odd index in the internal array). This is very cache-friendly for small maps.

Q22. What is the symmetry requirement of equals() and why is it hard to maintain with inheritance?

If a.equals(b) then b.equals(a). With inheritance: if ColorPoint extends Point and Point.equals() ignores color, then new Point(1,1).equals(new ColorPoint(1,1,RED)) may be true but new ColorPoint(1,1,RED).equals(new Point(1,1)) may be false (if ColorPoint checks color). Use composition over inheritance for value types.

Q23. How do you safely iterate a ConcurrentHashMap while modifying it?

ConcurrentHashMap's iterators are weakly consistent — they reflect some state of the map at some point during iteration. They never throw ConcurrentModificationException. Modifications during iteration may or may not be visible. forEach(threshold, action) is the preferred parallel-safe iteration method.

Q24. What is the difference between replaceAll() and forEach() on a Map?

forEach(BiConsumer) iterates entries for side effects, cannot modify values through the entry. replaceAll(BiFunction) replaces every value in-place with the function result — legal modification during iteration because it goes through the Entry.setValue() path.

Q25. How would you design a thread-safe cache with expiry and bounded size?

For production: use Caffeine (maximumSize, expireAfterWrite/Access). For a simple solution: ConcurrentHashMap for thread-safe storage + ScheduledExecutorService for TTL expiry + AtomicInteger for size control. Avoid LinkedHashMap-based LRU in concurrent context — it requires full synchronization.

Summary Chapter Summary & Quick Reference

This companion chapter explored Map implementations at a deeper level. HashMap's Node[] table, power-of-2 capacity, XOR bit-spreading in hash(), and resize without full rehashing give O(1) average performance. The equals()/hashCode() contract must be fully upheld: if equals() is true, hashCodes must match; override both or use records. Bad hashCode() degrades HashMap to O(n); Java 8 treeification limits this to O(log n). ConcurrentHashMap advanced features — computeIfAbsent atomicity, parallel forEach/reduce, Striped64 size approximation — enable scalable concurrent applications. IdentityHashMap fills the niche of reference-equality keyed maps. Map.of()/Map.copyOf() provide truly immutable maps, unlike Collections.unmodifiableMap() which is merely a view wrapper. Mutable keys, null handling differences, iteration order assumptions, and non-atomic compound operations are the four most common Map pitfalls.

TIP: The most important interview takeaway: HashMap uses hashCode() to find the bucket, then equals() to find the exact entry. Both methods must be consistent and cover the same fields. When in doubt, use records — they get both right automatically.